

How do Time Travel Debuggers Work?

~~Greg Law~~

Mark Williamson

**Most programmers
spend most of their
time debugging.**



*Everyone knows that debugging
is twice as hard as writing a
program in the first place.*

*So if you're as clever as you can
be when you write it, how will
you ever debug it?*

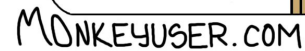
Brian Kernighan



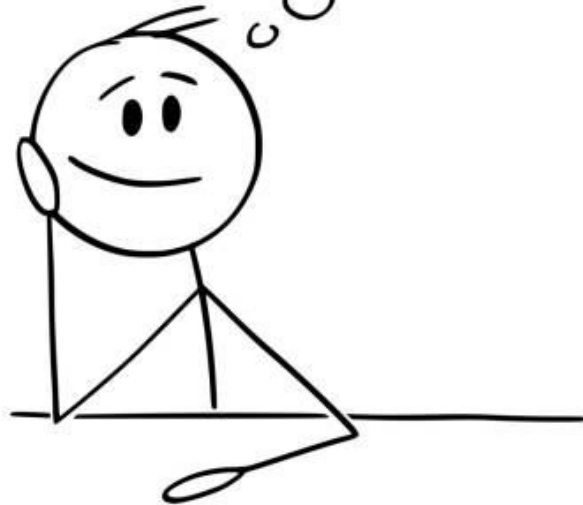
"MY printf's COMPILING."



STEP BY STEP DEBUGGING



How did that happen?



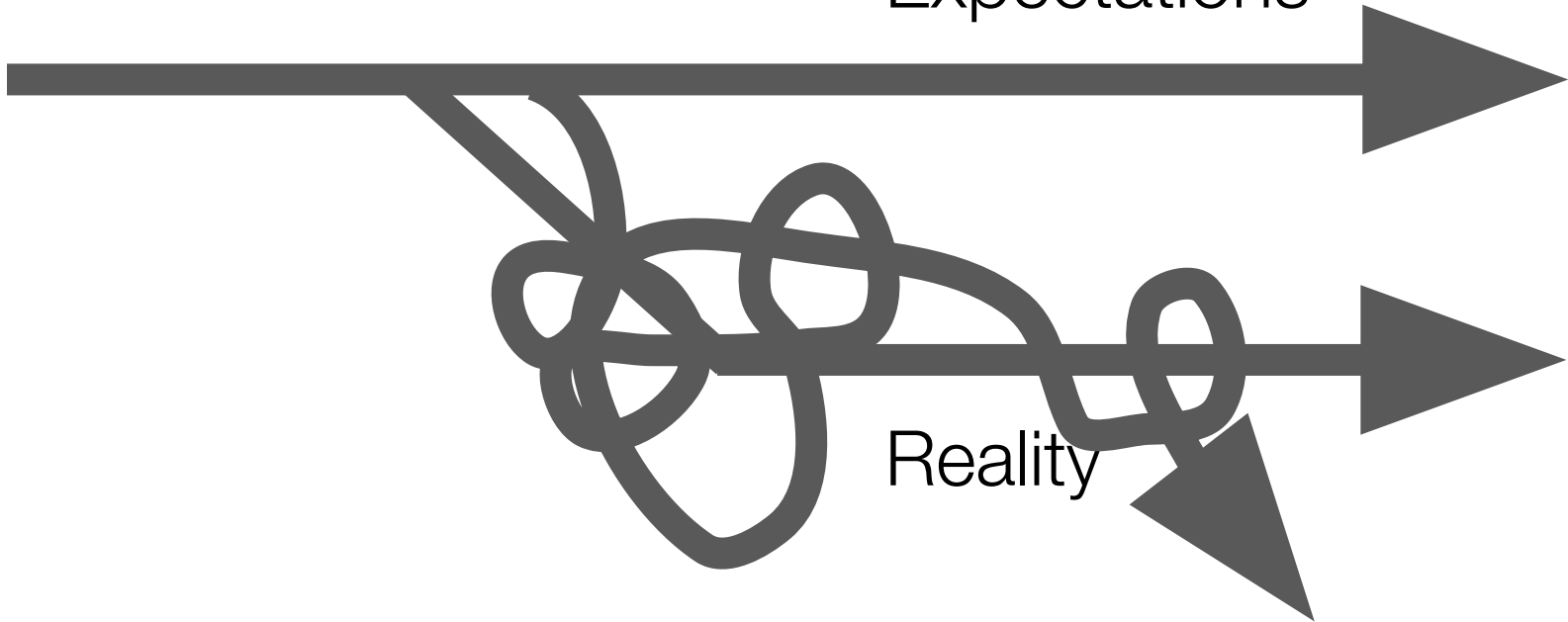


unc



Expectations

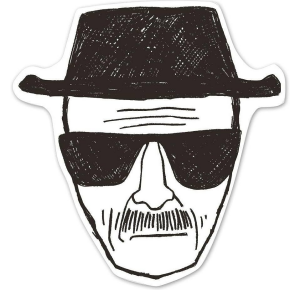
Reality





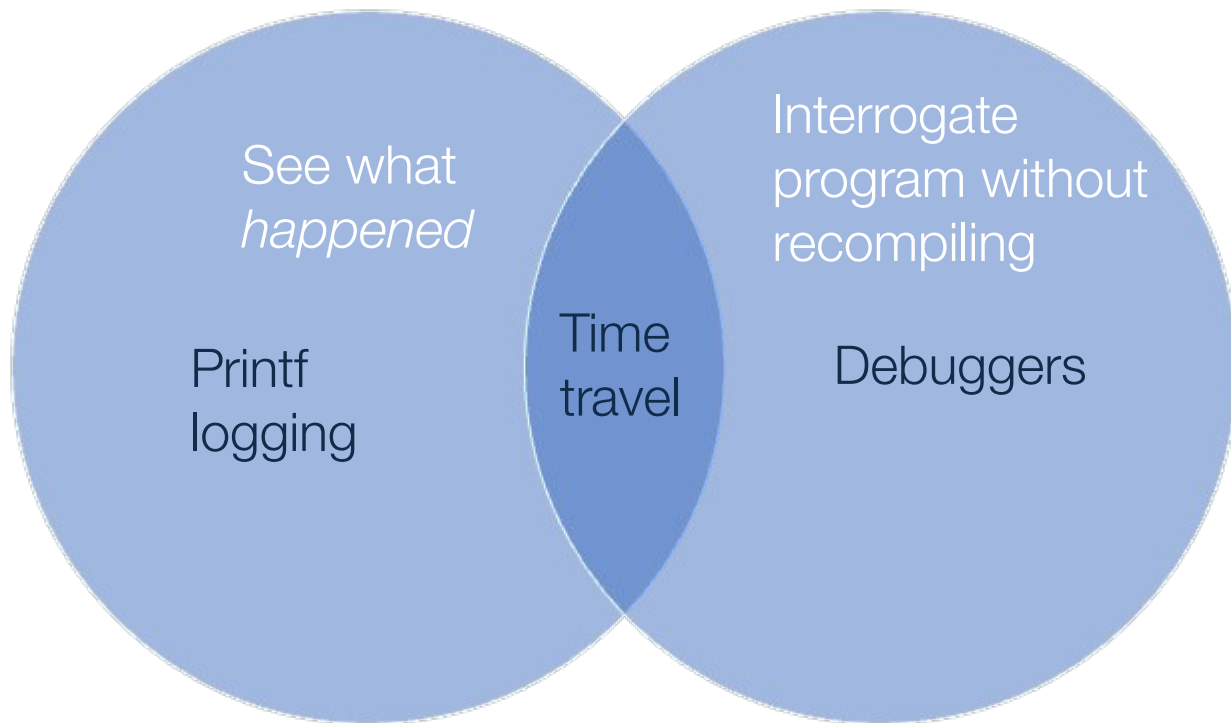


What makes bugs hard to fix?



‘Heisenbugs’

What happened... in detail





C++ projects and products

Linux

- Undo - UDB & LiveRecorder
- rr (rr-project.org)
- GDB (ish)

Windows

- TTD

Embedded

- Lauterbach “TRACE32”
- Green Hills TimeMachine

undo™



Non C++

- JavaScript / React replay.io
- .Net RevDebug
Visual Studio
- Java Undo
- Rust, Go Undo / rr
- Python Undo*, PyPy*

Building a time machine!

- How to remember?
 - What happened? How to repeat it?
- What exactly to record?
 - What's the right boundary?
- How to track time?
 - In what units? How do we count them?
- Rely on memory determinism?
 - What can we trust to stay the same?

Design Decision 1.

How to remember

What was the previous state?

Two options:

1. Save it.
2. Recompute it.

$$a = a + 1$$

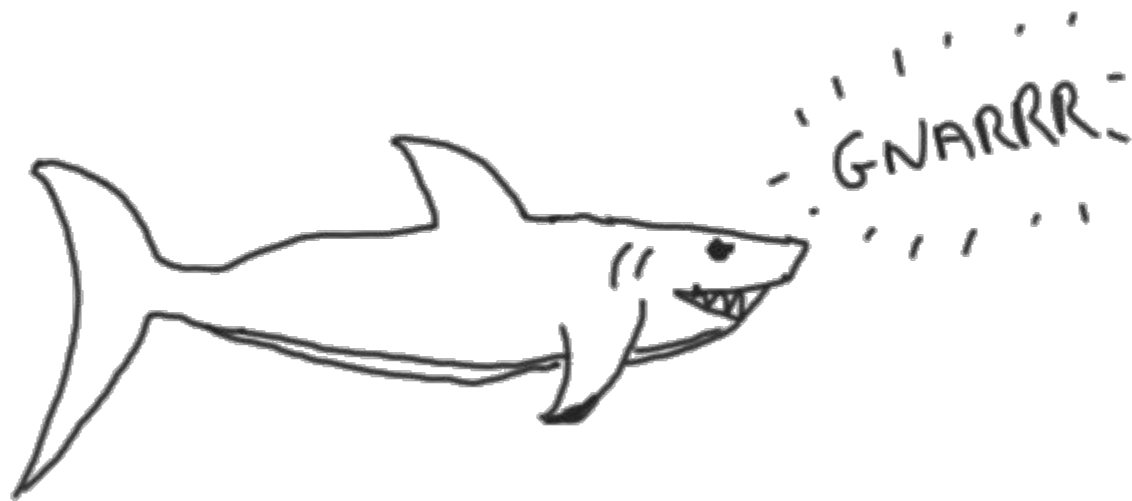
$$\Rightarrow a = b$$

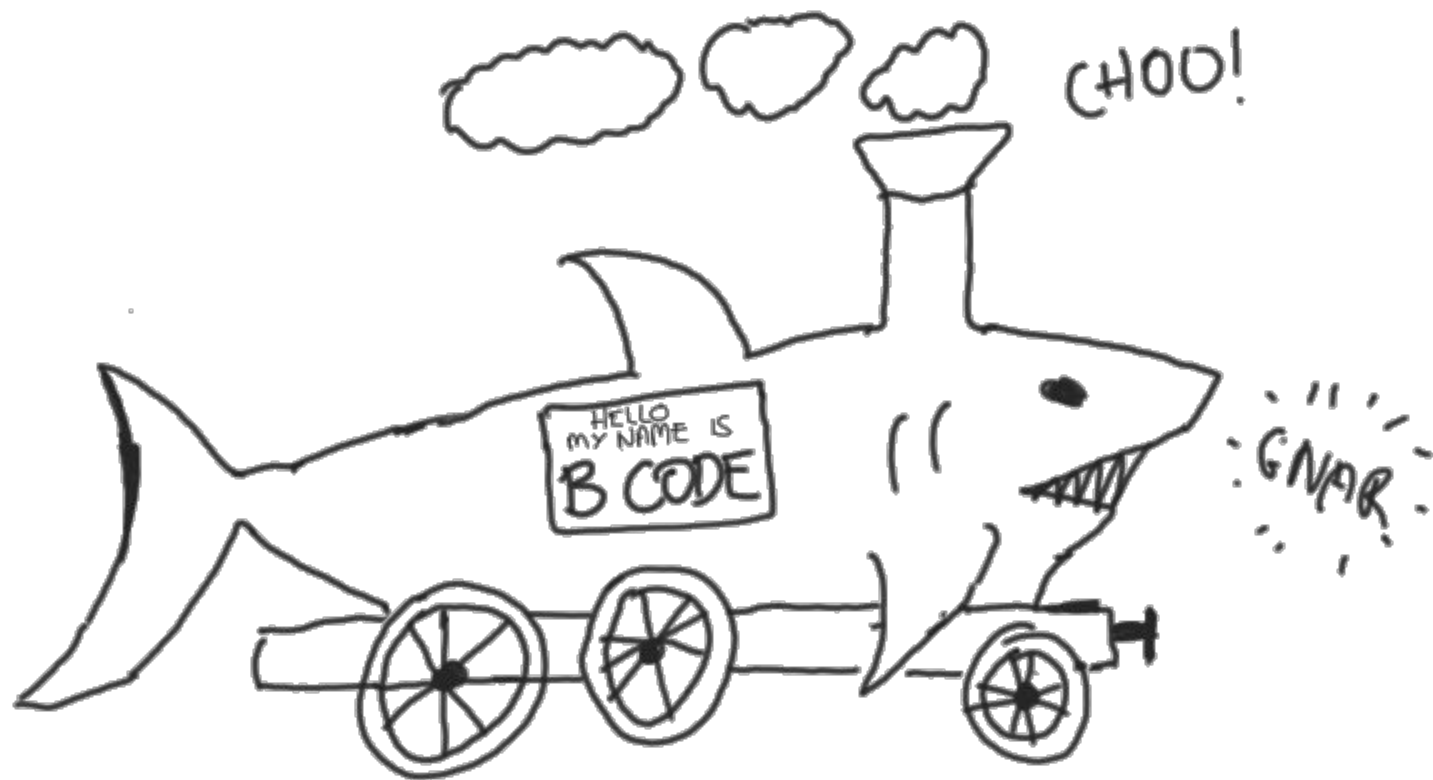
Deterministic re-execution

$$f(x) \rightarrow x'$$









Deterministic re-execution

- Snapshots.
- Event log.

Non-determinism

- What is unpredictable?
 - ~~System calls.~~
 - Thread switches.
 - Asynchronous events (signals).
 - Shared memory accesses.
 - Some machine instructions.

Non-deterministic code

0x00005555555515e	mov -0x4(%rbp),%eax
0x000055555555161	mov %eax,%esi
0x000055555555163	lea 0xe9a(%rip),%rdi
0x00005555555516a	mov \$0x0,%eax
0x00005555555516f	sub %rdx,%rsp
0x000055555555171	mov %rcx,%r10
0x000055555555174	mov \$0x11,%eax
0x000055555555179	syscall

Here's some machine code in a program that we want to be able to **record** and **replay**.

deterministic

nondeterministic

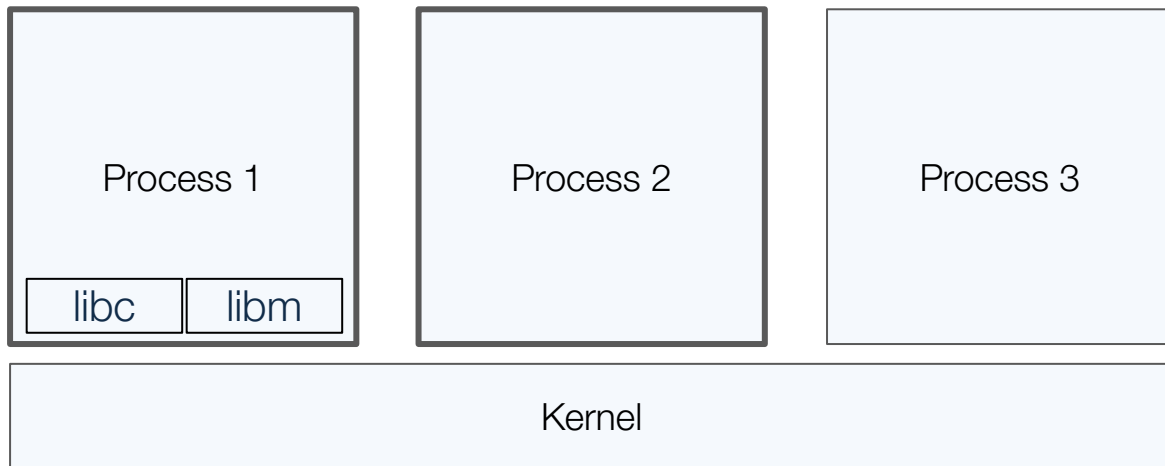
Design Decision 1

Re-execution is the winner

Design decision 2: what exactly to record?

- A process?
- A thread or function?
- A sandbox environment (e.g. JVM or v8?)
- Virtual machine?

Recording at process/OS ABI boundary



Design Decision 2 - winner!

Re-execution at the process boundary is the winner

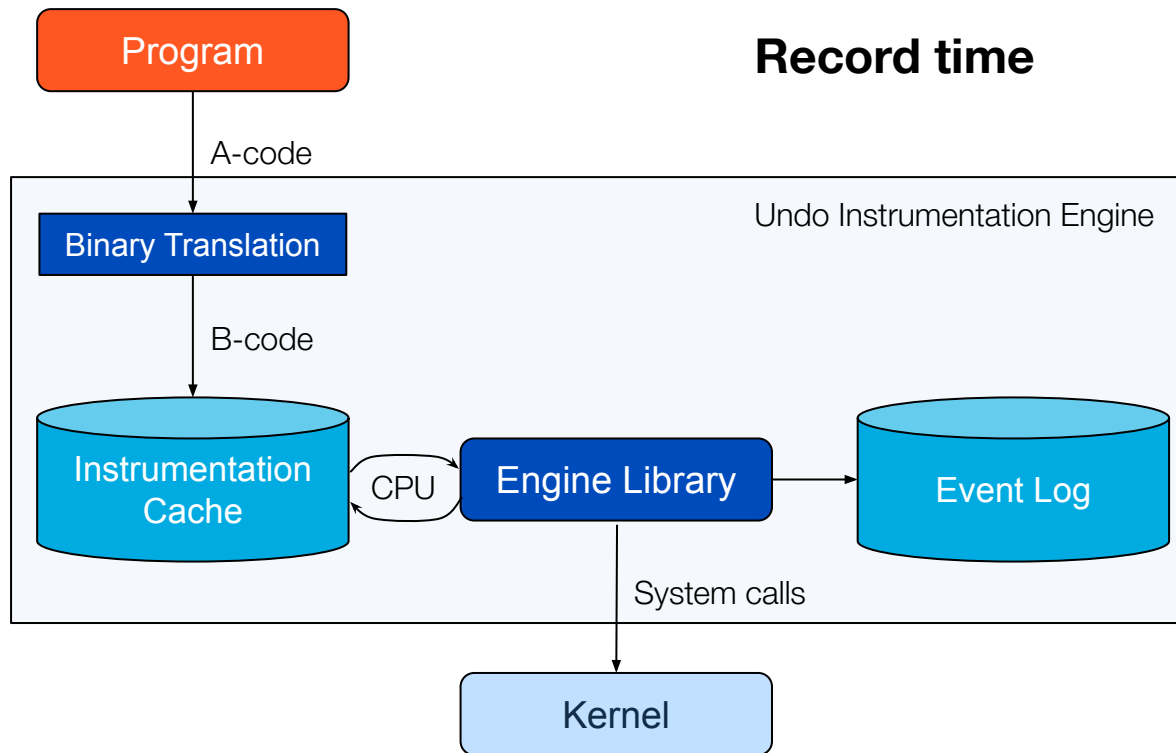
Design Decision 3: How to track time?



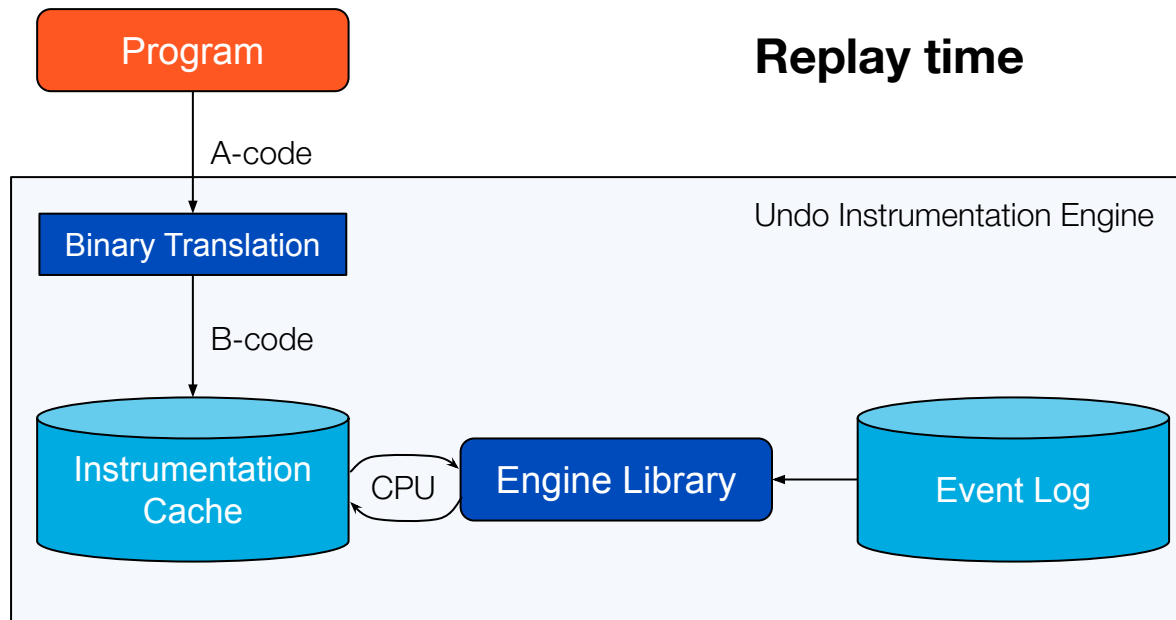
Design Decision 3: How to track time?

To JIT or not to JIT?

Translation and events



Translation and events



Perf counters

- Can be very fast.
- Simpler (less code).

JIT

- Works anywhere
- With anything*.
- Solid

Design Decision 4: Rely on memory determinism?

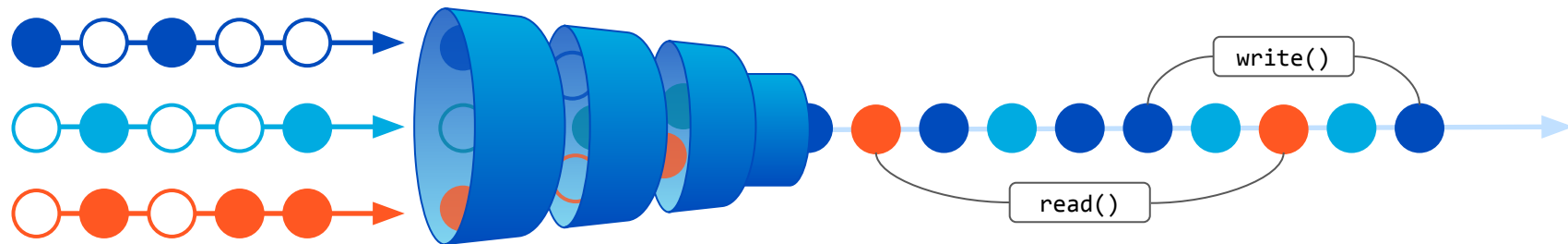
Memory Determinism

- (Much) smaller event logs
- Faster single-threaded

Not

- Unknown syscalls
- Parallel record

Thread serialization



Non-determinism

- What is unpredictable?
 - ~~System calls.~~
 - Thread switches.
 - Asynchronous events (signals).
 - ~~Shared memory accesses.~~
 - Some machine instructions.

Design decisions

- At what boundary to capture
- Binary rewriting instrumentation
- All/some/no memory recording
- Separate record/replay phases
- Parallel thread recording

Undo	rr	WinDbg	replay.io	ODB
proc	proc	proc	proc	JVM
yes	no	yes	no	no
some	none	all*	none	all
yes/no	yes	yes	yes	yes
no	no	yes	no	yes

Debuginfo

```
(gdb) print foo  
<value optimized out>  
(gdb)
```